❖ Array

- An array is a special type of variable used to store multiple values of same data type at a time.

Or

- An array is a collection of similar data items stored in continuous memory locations with single name.

Or

- An array is a collection of elements of the same data type.

Example1 : int a[5] = {10,20,30,40,50};
Output : 10 20 30 40 50

Example2 : int a[5] = {10,'A',20.5,50,60};
Output : 10 65 20 50 60

(use Implicit Type Conversion / type casting)

Here same data type means – same storage data type

- All Elements are stored in the Contiguous memory.

❖ Declaration of an Array

We use the following general syntax to create an array...

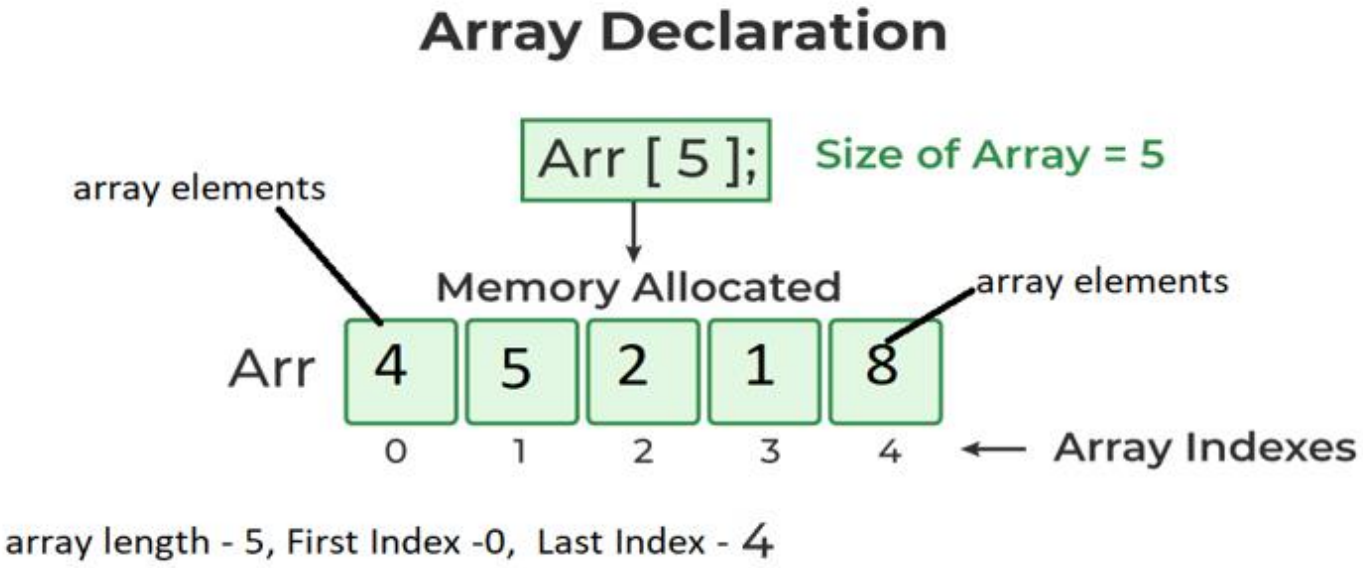
```
datatype arrayName [ size ] ;
```

Syntax for creating an array with size and initial values

```
datatype arrayName [ size ] = {value1, value2, ...} ;
```

Syntax for creating an array without size and with initial values

```
datatype arrayName [ ] = {value1, value2, ...} ;
```



array length - 5, First Index -0, Last Index - 4

❖ Array Terminologies

- **Size /length**: Number of elements or capacity to store elements in an array. It is always mentioned in square brackets [ ].
- **Type**: Refers to data type. It decides which type of element is stored in the array. It is also instructing the compiler to reserve memory according to the data type.
- **Base**: The address of the first element is a base address. The array name itself stores address of the first element.
- **Index**: The array name is used to refer to the array element.For example num[x], num is array and x is index. The value of x begins from 0.The index value is always an integer value.
  - ✓ Formula find index value = n-1 (n= length/size of array)
  - ✓ First index of any array is always 0 (zero).
  - ✓ Last index = n-1 (n = length of array)
- **Range:** Value of index of an array varies from lower bound to upper bound. For example in num[100] the range of index is 0 to 99.

❖ Rules of Array declaration in C++

1. Square brackets[] are required when declaring an arrya.
2. The general syntax for declaring an array is : data_type array_name [size];
3. The array size can be specified inside square brackets. Example : int a[5];
4. All the elements of the array share a common name .

HIRA KUMAR

5. Square brackets can be left empty when the array is initialized with values. Example : int a[] = {10,20,30};
6. An array can store multiple elements of the same data type.
7. Arrays required more bluk store data not use for 2-3 limited data. Use 5 or more than data store in same variable then use array.
8. When we declare an array with a specified size, memory is allocated for that number of elements.
   int a[5] = {10,20,50,60,78}; // output : 10 20 50 60 78
   This array can store a maximum of 5 integers.
9. If we declare an array of size 5 but initialize it with only 2 elements, the remaining elements are initialized to zero.
   int a[5] = {10,20}; // output : 10 20 0 0 0
10. If we declare an array of size 5 but try to initlialize it with more than 5 elements, the declaration is invalid.
    int a[5] ={10,50,60,78,96,45,25,36,45} // output : syntax error
11. If we omit the array size and initialize the array with values, the compiler determines the size automatically.
    int a[] ={20,45,96} // output 20 45 96

SI.No.	Example	Array ?
1.	int a [3] ;	Valid (yes)
2.	int hira[5]={10,20,30,40,50};	Yes
3.	int hira[]={10,20,50,30,80,90,45};	Yes
4.	char c[3]={'R', 'A','M'};	yes
5.	int [] = {1,5,10,20,45};	No (missing array name)
6.	int a[] = {1,30,2.3,55};	No (different data type value)
7.	char a[] = {1 , A , B };	No (Missing single quotes)
8.	int hira = { 10,20,50,40};	No (Missing square bracket)
9.	int a[3] = { 10,20,30}	No (Missing semicolon)
10.	int a[]= (10,20,30,40);	No (use curly braces not round brackets)
11.	int a[5]={10,20};	Yes
12.	int a[3] ={10,50,30,45,60};	No (Elements exceeds/more)
13.	int [5] hira = { 10,20,50,60,70}	No (size declaration after array_name).
14.	int a[5] = {1, 10, 10, 20.3, 3};	No error (type casting use)

		Output : 1 10 10 20 3
15.	int a[5] ={1,'A',49,20,56}	No error (type casting use) Output : 1 65 49 20 56
16.	double a[3] = {20,2.5,'A'}	No error (type casting use) Output : 20.0 2.5 65.0

❖ Characteristics of an array:

1. The declaration int a [5] is nothing but creation of five variables of integer types in memory instead of declaring five variables for five values.
2. All the elements of an array share the same name and they are distinguished from one another with the help of the element number.
3. The element number in an array plays a major role for calling each element.
4. Any particular element of an array can be modified separately without disturbing the other elements.
5. Any element of an array a[ ] can be assigned or equated to another ordinary variable or array variable of its type.
6. Array elements are stored in contiguous memory locations.

❖ Advantages & disadvantages

Advantages	Disadvantages
Easier to store elements of same data type	Wastage of memory because homogeneous/ contiguous memory locations

Q1. Write a C++ program to declare an integer array of 5 elements and print all the elements.

```cpp
#include <iostream>
#include<conio.h>
using namespace std;

int main()
{
int hira[5]={10,20,30,50,98};
cout << hira[3] <<endl;
cout << hira[0]<<endl;
cout << hira[5]<<endl; //garbage value
cout << "print all index value : ";
for (int i=0;i<=4;i++)
{
    cout << "  " << hira[i];
}
    getch();
}
```

```
C:\Users\hira\Desktop\hello.exe
50
10
612
print all index value :   10  20  30  50  98
```

Q2. Write a C++ program to input 5 integers from the user, store them in an array, and display all the elements.

```cpp
#include <iostream>
#include<conio.h>
using namespace std;

int main()
{
int hira[5];
cout << "Enter any five value : ";
for (int a=0;a<=4;a++)
{
    cin>> hira[a];
}
cout << "print all elements value : ";
for (int i=0;i<=4;i++)
{
    cout << "  " << hira[i];
}
    getch();
}
```

```
C:\Users\hira\Desktop\hello.exe
Enter any five value : 20
40
60
80
90
print all elements value :   20  40  60  80  90
```

Q3. Write a C++ program to store students' names and their marks using arrays and display each student's name along with their marks.

```cpp
#include <iostream>
#include<conio.h>
using namespace std;

int main()
{
string a[]={"Hira","Ram","Shyam","Radha","Mohan"};
int b[]={75,80,60,95,90};
cout << "   " <<"Name" << "    "   << " Marks" << endl;
cout << "============================"<< endl;
for (int i=0;i<=4;i++)
{
    cout << "   " << a[i] << "    " << b[i] << endl;
}
    getch();
}
```

```
C:\Users\hira\Desktop\hello.exe
  Name     Marks
====================
  Hira    75
  Ram    80
  Shyam    60
  Radha    95
  Mohan    90
```

Q4. WAP in C++ program to input three numbers from the user, store them in an array, and calculate their sum.

```cpp
#include <iostream>
#include<conio.h>
using namespace std;

int main()
{
int ask[3],sum=0;
cout << " Enter any three number : ";
for (int a=0;a<=2;a++)
{
    cin >> ask[a];
}
for (int i=0;i<=2;i++)
{
    sum = sum+ask[i];
}
cout << " SUM = " << sum;
    getch();
}
```

```
C:\Users\hira\Desktop\hello.exe
 Enter any three number : 10
20
50
 SUM = 80
```

Questions

1. Write a C++ program to input 5 integers into an array and print them in reverse order.

2. Write a C++ program to input 5 integers into an array and calculate their sum.

3. Write a C++ program to input 5 integers into an array and calculate their average.

4. Write a C++ program to input 5 integers into an array and find the largest element.

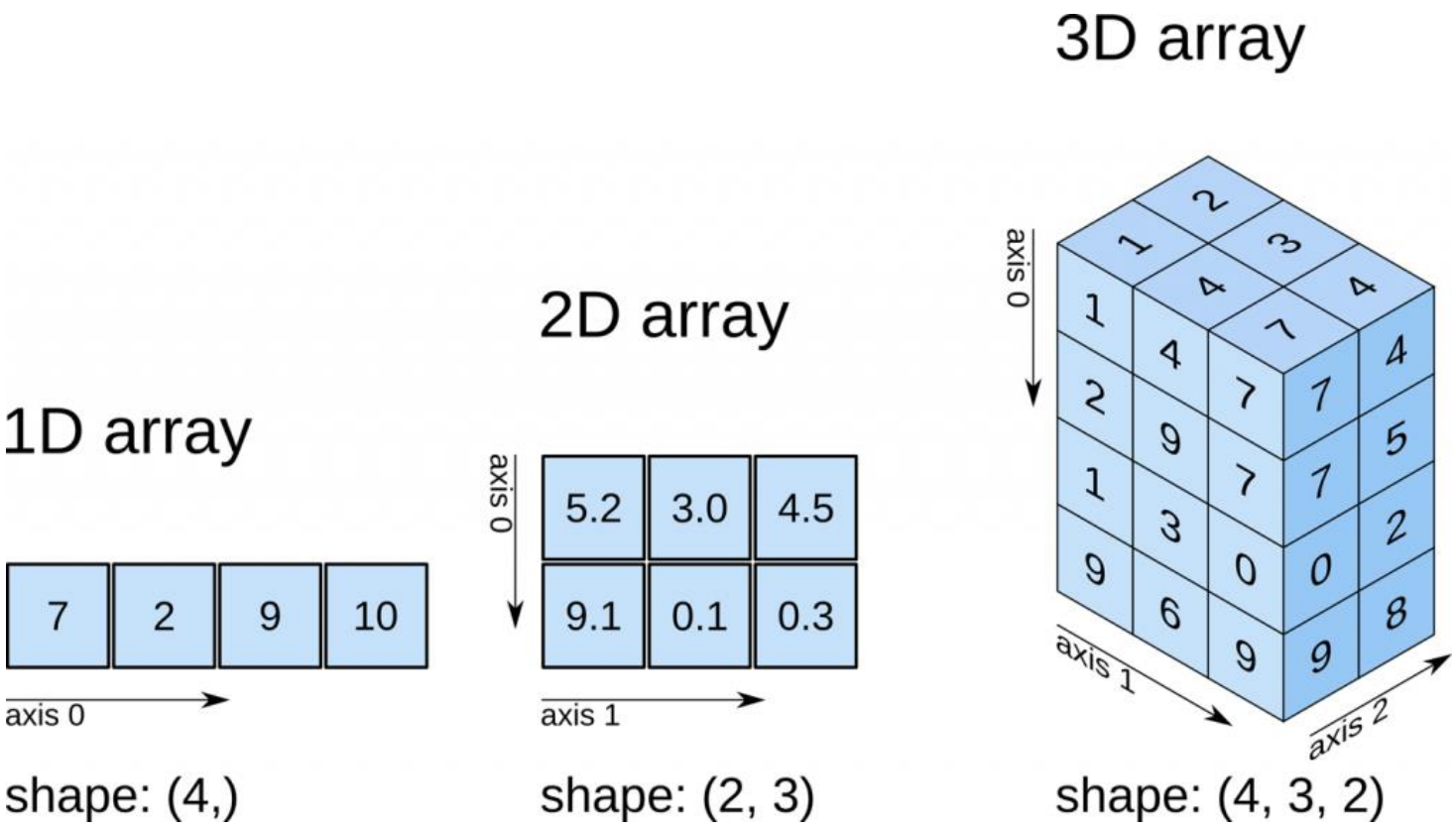
5. Write a C++ program to input 5 integers into an array and find the smallest element.

HIRA KUMAR

❖ Types of Array

Arrays are classified as:

1. Single dimensional array / One Dimensional Array (1D)
2. Multidimensional array
   a) Two – Dimensional
   b) Three – Dimensional



1. Single dimensional array(1D)

- Single or One Dimensional array is used to represent and store data in a linear form.
- Array having only one subscript variable is called One-Dimensional array.
- It is also called as Single Dimensional Array or Linear Array.

Single Dimensional Array Declaration and initialization:

- Syntax for declaration: [size];
  ✓ Examples for declaration: int iarr[3]; char carr[20]; float farr[3];
- Syntax for initialization: [size] = {val1, val2, …, valn};
  ✓ Examples for initialization: int iarr[3] = {2, 3, 4}; char carr[20] = “program”; float farr[3] = {12.5, 13.5, 14.5};

Methods of Initializing 1-D Array

The various ways of initializing arrays are as follows:

1. Initializing all elements of array(Complete array initialization)
2. Partial array initialization
3. Initialization without size.
4. String initialization

1. Initializing all elements of array

- Arrays can be initialized at the time of declaration when their initial values are known in advance.
- In this type of array initialization, initialize all the elements of specified memory size.
- Example:

```
int a[5]={10,20,30,40,50};
```

10	20	30	40	50

2. Partial array initialization

- If the number of values to be initialized is less than the size of array then it is called as partial array initialization.
- In such a case elements are initialized in the order from 0th element.
- The remaining elements will be initialized to zero automatically by the compiler.
- Example:

```
int a[5]={10,20};
```

10	20	0	0	0

3. Initialization without size

- In the declaration the array size will be set to the total number of initial values specified.
- The compiler will set the size based on the number of initial values.
- Example:

```
int a[ ]={10,20,30,40,50};
```

In the above example the size of an array is set to 5

4. String Initialization

- Sequence of characters enclosed within double quotes is called as string.
- The string always ends with NULL character(\0).

S	V	I	T	\0
S[0]	S[1]	S[2]	S[3]	S[4]

Write a program in c++ to programmer manually/self initialize array (1D) and print array values.

hira.cpp

```
#include<iostream>
#include<conio.h>
using namespace std;
int main()
{
   int i=0;
    int array[5] = {10,20,30,40,50};
    while(i<=4)
    {
    cout << array[i]<<endl;
    i++;
    }
    getch();
}
```

C:\Users\hira\Desktop\hira.exe

```
10
20
30
40
50
```

OR

hira.cpp

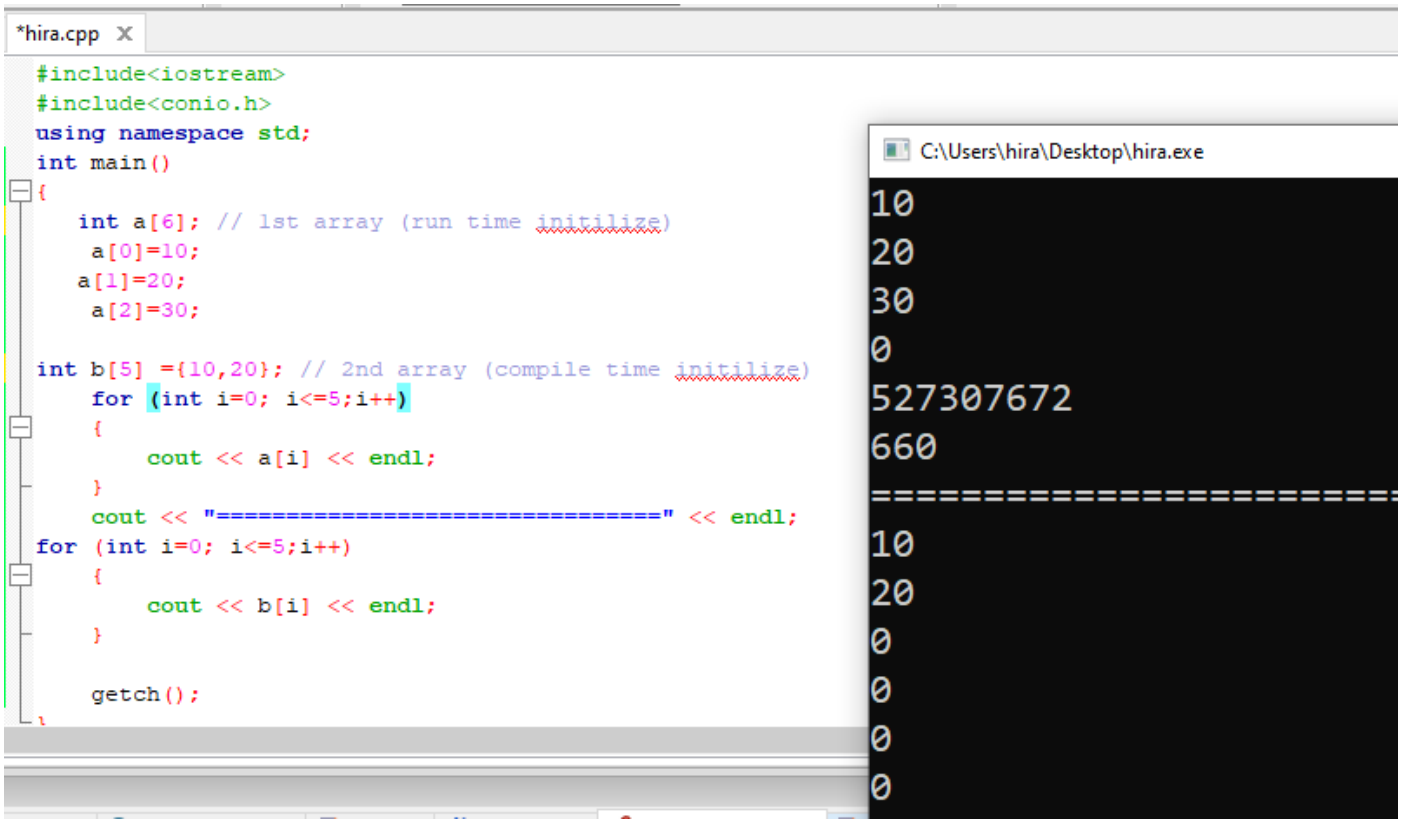
```
#include<iostream>
#include<conio.h>
using namespace std;
int main()
{
    int a[6];
     a[0]=10;
    a[1]=20;
     a[2]=30;
     a[3]=40;
     a[4]=50;
     a[5]=60;
     for (int i=0; i<=5;i++)
     {
         cout << a[i] << endl;
     }

     getch();
}
```

C:\Users\hira\Desktop\hira.exe

```
10
20
30
40
50
60
```

OR



2. Multidimensional Array

A multidimensional array is an array that contains two or more dimensions and is used to store data in tabular form, such as rows and columns.

Types of multidimensional array

(a) Two-dimensional array (2D)
(b) Three-dimensional array (3D)
(c) Higher-dimensional array

(a) Two-Dimensional array (2D)

- A two-dimensional array is an array that stores elements in the form of rows and columns.
- Syntax- type arr_name[m][n];
  where,
  - **type:** Type of data to be stored in each element.
  - **arr_name:** Name assigned to the array.
  - **m:** Number of rows.
  - **n:** Number of columns.
- Example →

  int arr[3][4] = {0, 1 ,2 ,3 ,4 , 5 , 6 , 7 , 8 , 9 , 10 , 11} **OR**

int A [5] [4]; **OR**

int A [2] [4] = {1, 2, 3, 4, 5, 6, 7, 8}; **OR**

int A [2] [4] = { {1, 2, 3, 4 }, {5, 6, 7, 8} };

	Column 0	Column 1	Column 2
Row 0	x[0][0]	x[0][1]	x[0][2]
Row 1	x[1][0]	x[1][1]	x[1][2]
Row 2	x[2][0]	x[2][1]	x[2][2]

-

Numerical questions formula

- **Find row & column range**
  - ✓ In C/C++, arrays are 0-indexed, so the row number ranges from 0 to (m-1) and the column number ranges from 0 to (n-1).
  - ✓ Example : int arr[3][4] = {0, 1 ,2 ,3 ,4 , 5 , 6 , 7 , 8 , 9 , 10 , 11}
  - ✓ Row range : 0 to 2, & column range : 0 to 3

Data Type	Size
Int	4 bytes
Long	8 bytes
Float	4 bytes
Double	8 bytes
Bool	1 byte (true/false)
Char	1 bytes
Short	2 bytes

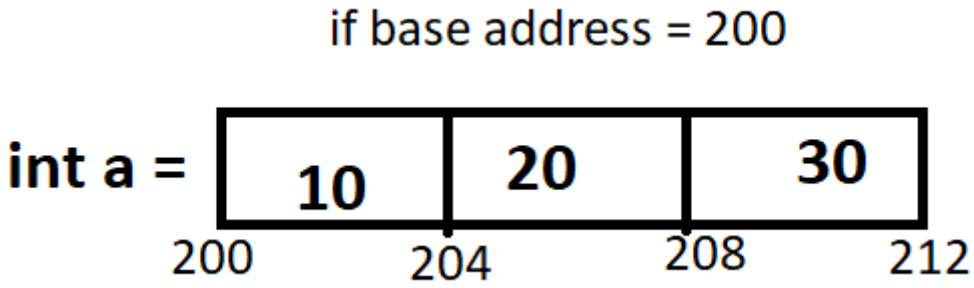
❖ Array in memory

➢ Consider this array (1D)

int a[3] = {10,20,30}

→ here, data type is **int** which contain memory **4 byte**/element.

In this array total 3 element, So total memory contain/occupied 4*3=12 byte .



➢ In 2d array

A[0][0] 1000	A[0][1] 1004	A[0][2] 1008	A[0][3] 1012
A[1][0] 1016	A[1][1] 1020	A[1][2] 1024	A[1][3] 1028
A[2][0] 1032	A[2][1] 1036	A[2][2] 1040	A[2][3] 1044

Formula : Find address of 2D array

→ **BA +S[(TR-i)*N+(TC-j)]**

where:

- BA = Base Address
- S = size of every element(in bytes)
- TR = target row
- TC = target column
- i = starting Row index
- j = starting Column index
- N = total number of columns only

Example : See Q10.

➢ Solved Question / Example

Q1. int arr[2][5]; में कितने elements होंगे ? (2D array)

i) 2 ii) 5 iii) 7 iv) 10 v) 11

Ans – iv)10 [Reason → Find element formula = m*n → 2*5 →10]

Q2. int arr [2][5]; को कितना bytes allocate होगा ?

i) 20 ii) 50 iii) 40 iv) 10 v) 30

Ans – iii)40 [Reason → Find bytes formula=(m*n*size of datatype)]

= 2*5*4→ 40

Q3. float arr [2][5]; को कितना bytes allocate होगा ? (2d Array)

i) 20 ii) 50 iii) 40 iv) 10 v) 30

Ans – iii) 40 [2*5*4=40]

Q4. int age []=new int[5], how many bytes will required by marks array. ? (1D array → Java base array)

i) 20 ii) 50 iii) 40 iv) 10 v) 30

Ans – i) 20 [Reasong – 4*5 = 20]

Q5. How many values can be held by an array A(-3:10,5:12)

(एक सारणी A (-3:10,5:12) में कितने मान रखे जा सकते हैं)

i) 112 ii) 91 ii) 119 iv) 110

Ans – i) 112

Explain

- -3:10 represents the first dimension (Rows).
- 5:12 represents the second dimension (Columns).

Step 1: Calculate the Number of Rows

The first dimension ranges from -3 to 10.

Indices: -3, -2, -1, 0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10

Formula – Find no of row/column:

Rows/column = U-L+1

U = Upper Bound → Last index/value

L = Lower Bound → First index/value

So,

Total Row =10-(-3)+1=14

Therefore, the array has 14 rows.

Step 2: Calculate the Number of Columns

The second dimension ranges from 5 to 12.

Indices: 5, 6, 7, 8, 9, 10, 11, 12

So,

Total Columns = 12-5+1=8
Therefore, the array has 8 columns.

Step 3 : Here, A {14,8}

जब dimension (Row & Column) दिया हो तो array का साइज़ (Total value/element) = m∗n मतलब = 14∗8 = 112

Q6. How many Values can be held by an array A (-4:9, 6:11)?

Ans - 84

Q7. Considering the size of float variable as four bytes, if base address of an array of float is 100, then the address of 4th cell is?

(फ्लोट variable के आकार को 4 bytes मानते हुए, यदि float की एक सारणी का आधार पता 100 हैं, तो चौथी सेल का पता (address) हैं ?

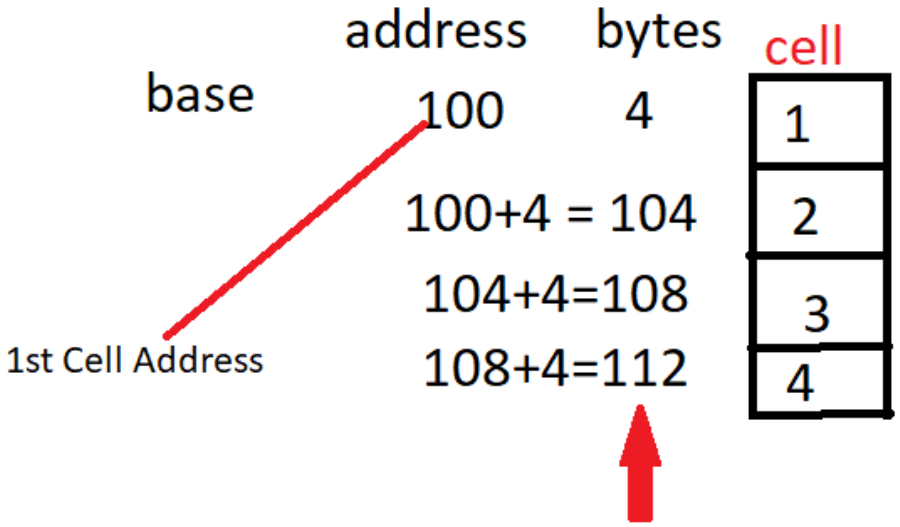
i)116 ii) 104 iii) 112 iv) 100 v) 400

Ans – iii)112

Given – Float = 4 bytes

Base Address = 100

Address of 4th cell = ?



```
#include <iostream>
#include<conio.h>
using namespace std;

int main()
{
int ask[3];
cout << " Enter any three number : ";
for (int a=0;a<=2;a++)
{
    cin >> ask[a];
}
for (int i=0;i<=2;i++)
{
    cout << " Address of each cell :  ask["<< i<<"] = " << &ask[i] << endl;
}

    getch();
}
```

```
C:\Users\hira\Desktop\hello.exe
 Enter any three number : 20
48
96
 Address of each cell :  ask[0] = 0xa5657ffe1c
 Address of each cell :  ask[1] = 0xa5657ffe20
 Address of each cell :  ask[2] = 0xa5657ffe24
```

```
Blocks X   Search results X
Line   Message
       === Build
       === Build
```

Q8. Considering the size of float variable as four bytes, if base address of an array of float is 500, then the address of 3rd cell is?

i)500 ii) 508 iii) 512 iv) 199 v) 505

Ans – ii) 508

Q9. Considering the size of float variable as 2 bytes, if base address of an array of float is 300, then the address of 5th cell is?

i)300 ii) 302 iii) 306 iv) 308 v) 505

Ans – iv)308

Q10. Consider a 2D character array A[1:5,2:7]. Assuming row major order of storage determine the address of element A[2][3] if base address is 500 and each element contain 1 byte?

(एक 2D array सारणी A[1:5,2:7] पर विचार करें | भंडारण के पंक्ति प्रमुख क्रम को मानते हुए तत्व A[2][3] का पता निर्धारण करें यदि आधार पता 500 हैं ?)

i)501 ii) 506 iii) 507 iv) 508 v) 500

Ans – iv) 508

यहाँ A[1:5,2:7]

Starting Row (i) = 1, Ending Row = 5

Starting Column (j) = 2, Ending Column = 7
Total Columns (N) = 7 − 2 + 1 = 6
S = 1 byte
→ A [5-1+1,7-2+1]→A[5,6]
Findd A[2][3] = A [TR][TC] (मतलब Target Row =2, Target Column j=3)
Base = 500
i = 1 , j = 2 , S = 1
Formula → **BA +S[(TR-i)*N+(TC-j)]**
500 + 1(2-1)*6+(3-2)
500+1*1*6+1
500+6+1
507 (Answer)

index

Row ↓ / Column →	2	3	4	5	6	7
1	A[1][2]	A[1][3]	A[1][4]	A[1][5]	A[1][6]	A[1][7]
2	A[2][2]	A[2][3]	A[2][4]	A[2][5]	A[2][6]	A[2][7]
3	A[3][2]	A[3][3]	A[3][4]	A[3][5]	A[3][6]	A[3][7]
4	A[4][2]	A[4][3]	A[4][4]	A[4][5]	A[4][6]	A[4][7]
5	A[5][2]	A[5][3]	A[5][4]	A[5][5]	A[5][6]	A[5][7]

Address

Row ↓ / Column →	2	3	4	5	6	7
1	500	501	502	503	504	505
2	506	507	508	509	510	511
3	512	513	514	515	516	517
4	518	519	520	521	522	523
5	524	525	526	527	528	529

Q11. Consider a 2D character array A[3:9,4:6]. Assuming row major order of storage determine the address of element A[4][5] if base address is 100?

i)100 ii) 112 iii) 113 iv) 115 v) 109

Ans – 113

HIRA KUMAR

Array MCQ

Q1. How many elements will be present in int arr[3][4]? (2D Array)

i) 7 ii) 12 iii) 16 iv) 10 v) 8

Q2. How many bytes will be allocated to int arr[3][6]? Assume int = 4 bytes.

i) 24 ii) 36 iii) 72 iv) 48 v) 60

Q3. How many bytes will be allocated to float arr[4][5]? Assume float = 4 bytes.

i) 20 ii) 40 iii) 80 iv) 60 v) 100

Q4. How many values can be held by an array A(1:10, 2:6)?

i) 40 ii) 50 iii) 60 iv) 30 v) 45

Q5. How many values can be held by an array A(-5:5, 1:4)?

i) 40 ii) 44 iii) 48 iv) 36 v) 32

Q6. How many values can be held by an array A(0:7, -2:3)?

i) 36 ii) 42 iii) 48 iv) 40 v) 56

Q7. Considering the size of an integer as 4 bytes, if the base address of an integer array is 200, what is the address of the 5th element? Assume the first element is at the base address.

i) 204 ii) 216 iii) 220 iv) 212 v) 224

Q8. Considering the size of a float as 4 bytes, if the base address of an array is 1000, what is the address of the 6th element? Assume the first element is at the base address.

i) 1020 ii) 1024 iii) 1028 iv) 1016 v) 1004

Q9. Consider a 2D character array A[1:4, 1:5]. Assuming row-major order, determine the address of element A[2][3] if the base address is 300. Assume each character occupies 1 byte.

i) 305 ii) 306 iii) 307 iv) 308 v) 309

Q10. Consider a 2D character array A[2:6, 3:7]. Assuming row-major order, determine the address of element A[4][5] if the base address is 500. Assume each character occupies 1 byte.

i) 509 ii) 510 iii) 511 iv) 512 v) 513

Answer Key (do match)

1. ii) 44
2. ii) 50
3. iii) 80

HIRA KUMAR

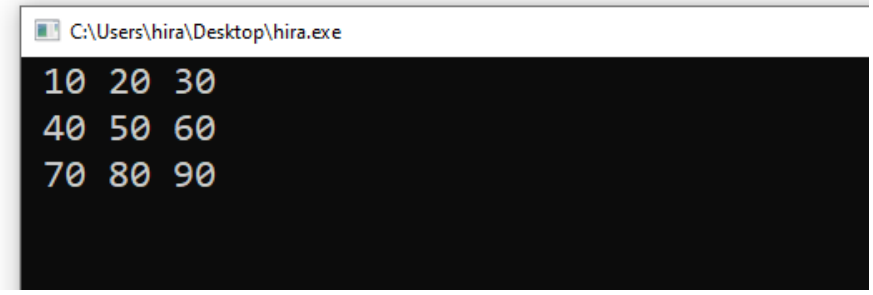
4. iii) 72
5. ii) 12
6. iii) 511
7. iii) 306
8. i) 1020
9. ii) 216
10. iii) 48

Q. Initialization of array elements at Compile-time

```
#include<iostream>
#include<conio.h>
using namespace std;
int main()
{
int A[3][3]= {{10,20,30},{40,50,60},{70,80,90}}; // declaration of Array where 3 row, and 3 column
int i,j;
for(i=0;i<3;i++) //1st for loop for row
{
for(j=0;j<3;j++) // 2nd for loop for column
{
cout << " " << A[i][j];
}
cout << endl;
}
getch();
   return 0;
}
```

hira.cpp

```cpp
#include<iostream>
#include<conio.h>
using namespace std;
int main()
{
int A[3][3]= {{10,20,30},{40,50,60},{70,80,90}}; // declaration of Array where 3 row, and 3 column
int i,j;
for(i=0;i<3;i++) //1st for loop for row
{
for(j=0;j<3;j++) // 2nd for loop for column
{
cout << " " << A[i][j];
}
cout << endl;
}
getch();
    return 0;
}
```



Q. Initialization of array elements at run-time

```cpp
#include<iostream>
#include<conio.h>
using namespace std;
int main()
{
int A[3][3]; // declaration of Array where 3 row, and 2 column
int i,j;
cout<< "Enter elements = " ;
for(i=0;i<3;i++) //1st for loop for row
{
for(j=0;j<3;j++) // 2nd for loop for column
cin>>A[i][j];
}
for(i=0;i<3;i++) //1st for loop for row
{
for(j=0;j<3;j++) // 2nd for loop for column
{
cout << " " << A[i][j];
}
cout << endl;
}
getch();
   return 0;
}
```

hira.cpp

```cpp
#include<iostream>
#include<conio.h>
using namespace std;
int main()
{
int A[3][3]; // declaration of Array where 3 row, and 2 column
int i,j;
cout<< "Enter elements = " ;
for(i=0;i<3;i++) //1st for loop for row
{
for(j=0;j<3;j++) // 2nd for loop for column
cin>>A[i][j];
}
for(i=0;i<3;i++) //1st for loop for row
{
for(j=0;j<3;j++) // 2nd for loop for column
{
cout << " " << A[i][j];
}
cout << endl;
}
getch();
    return 0;
```

```
C:\Users\hira\Desktop\hira.exe
Enter elements = 25
41
63
45
65
89
25
41
36
 25 41 63
 45 65 89
 25 41 36
```

==========HIRA KUMAR==========